

Name: Joshua Bourton

User-name: xtzv61

Algorithm A: IA (A* search with iterative deepening)

Algorithm B: BG (Basic Greedy Search)

Description of enhancement of Algorithm A:

Enhancement 1 (modification of the `prims_heuristic()` function): The final city in the minimum spanning tree (MST) (calculated from city zero) was used as the starting city, as opposed to a randomly generated one. This was done by modifying the algorithm used for calculating the MST heuristic value from a given node so that it also had the ability to return the MST itself. A boolean value was used to inform the function whether to return the MST itself or its heuristic value. The MST connects cities in a way that minimises the path cost, so selecting this final city helped provide a consistently low score for all ten provided city files.

Enhancement 2: Memoization was used inside the IDA* algorithm function `IDAStarTSP()` to memoize the heuristic values and thereby save valuable compute time.

This was done by creating a dictionary called `mst_heuristic_memo`, which serves as a cache to store previously computed heuristic values, preventing the recompute values which had already been found. During the depth-limited A* search performed inside the second while loop, the algorithm checks if the heuristic value has already been computed. If it has, its retrieved from the dictionary and reused, avoiding redundant calculations.

Description of enhancement of Algorithm B:

Enhancement 1A: To give the greedy algorithm a greater chance of returning a higher quality tour, a *multiple_restarts()* function was constructed that ran multiple iterations of the algorithm and returned the best tour found from amongst those run-throughs. To prevent the same starting cities being used for each iteration, the first ten edges in the sorted edge set in the *Greedy_TSP()* function were randomly shuffled.

Enhancement 1B: To prevent runtime from going beyond reasonable bounds, a function *select_num_restarts()* was written to select the number of times the *multiple_restarts()* function is allowed to run the greedy algorithm. It returns this value based on predefined numbers chosen from knowledge of how long the greedy algorithm takes to execute on the ten provided tours.

Enhancement 2: Inspired by the paper by Chaing, et al., (2016) (DOI [here](#)) Algorithm B was enhanced by adding a function to implement the 2-OPT technique. In this enhancement, the global *tour* variable was modified inside this algorithm to produce a tour with shorter path length.

2-OPT works by iterating over all the pairs edges in the tour and examining whether reversing the order of two cities in the tour would result in a tour with a lower path cost. If it would, the tour is subsequently modified to reflect the swap. Combined with enhancements 1A and 1B, this greatly improved tour quality, with the quality of tours found from the enhanced greedy algorithm on par with that of the iterative deepening A* search.

DESCRIPTION OF ALGORITHM ONLY IF THE ALGORITHM IS NOT COVERED IN LECTURES

Description of *non-standard* Algorithm A:

*Describe any non-standard algorithms you have implemented that **have not been covered in lectures** (otherwise these boxes should be blank) You need to convince me that your implementation is indeed that of the named algorithm and you need to **provide a full reference to the source for your algorithm**. You should **include a pseudocode description**. You can vary the sizes of these boxes but do not change the font (Calibri), font size (11), the paragraph properties (single space) or the header and footer and everything should fit onto one side of A4. (You can delete these instructions.)*

Remember: You need my express permission to implement a non-standard algorithm!

Description of *non-standard* Algorithm B: